Parallelization of 3D Discrete Element Method Solver for Spherical particle using OpenM

Shraddha Chowdhury
School of Physical Sciences
Indian Institute of Technology,Mandi
Himachal Pradesh, India
di2204@students.iitmandi.ac.in

***Abstract*—The Discrete Element Method (DEM) is a numerical technique used to simulate systems of interacting particles. The DEM simulation is implemented on shared-memory architectures and parallelized using OpenMP. The simulation is validated using three standard test cases, namely, gravity, integration accuracy, and contact dynamics. The results demonstrate that the parallel implementation produces physical consistent results with only minor numerical differences. The study highlights the importance of careful handling of shared data and numerical consistency in parallel simulations.**

***Index Terms*—DEM, parallelisation, OpenMP**

I. Introduction

The Discrete Element Method (DEM) is a computational technique used to simulate the dynamics of systems composed of discrete interacting particles. It is widely applied in fields such as granular physics, powder technology, geophysics, and material science.

In DEM, the motion of each particle is tracked individually by solving Newton's equations of motion, while interactions between particles are modeled through contact force laws. A key computational challenge in DEM arises from the evaluation of pairwise interactions, which scales as $O(N^2)$ with the number of particles N. This makes large-scale simulations computationally expensive.

To address this limitation, parallel computing techniques can be employed. In this work, OpenMP is used to parallelize the DEM simulation on shared-memory architectures. The goal is to improve computational performance while maintaining numerical correctness.

The study focuses on comparing serial and parallel implementations in terms of accuracy, stability, and execution time.

II. Mathematical Formulation

The motion of each particle is governed by Newton's second law:

$$m\frac{d\mathbf{v}}{dt} = \mathbf{F}$$

where $\mathbf{F}$ is the total force acting on the particle.

The total force consists of three contributions:

- Gravitational force
- Particle-particle contact force
- Particle-wall interaction force

The gravitational force is given by:

$$\mathbf{F}_g = mg\hat{z}$$

The contact force between particles is modeled as:

$$\mathbf{F}^c_{ij} = (k_n\delta_{ij} - \gamma_n v_{n,ij})\mathbf{n}_{ij}$$

where δ_{ij} is the overlap between particles, $v_{n,ij}$ is the relative normal velocity, k_n is the stiffness coefficient, and γ_n is the damping coefficient. The force is applied only when $\delta_{ij} > 0$.

The total kinetic energy of the system is defined as:

$$KE = \sum_{i=1}^{N} \frac{1}{2} m_i |\mathbf{v}_i|^2$$

which is used as a diagnostic quantity to monitor system evolution and verify correctness.

The system is evolved using an explicit time integration scheme. The simulation proceeds as follows:

1) Initializing particle positions randomly within the domain
2) Set initial velocities and forces to zero
3) For each time step:
   - Reset all forces to zero
   - Add gravitational force
   - Compute particle-particle contact forces
   - Compute wall interaction forces
   - Update particle velocities and positions

The integration scheme is given by:

$$\mathbf{v}_{t+\Delta t} = \mathbf{v}_t + \frac{\mathbf{F}}{m}\Delta t$$

$$\mathbf{x}_{t+\Delta t} = \mathbf{x}_t + \mathbf{v}_{t+\Delta t}\Delta t$$

This method is simple and computationally efficient, but requires a sufficiently small time step to maintain numerical stability.

III. Verification and Testing

To ensure the correctness and stability of the implementation, the simulation is validated using three standard test cases that isolate different physical components of the model, namely gravity, integration accuracy, and contact dynamics.

A single particle is allowed to fall under the influence of gravity without any interactions. The numerical trajectory is compared with the analytical solution of uniformly accelerated motion. As shown in Fig. 1, the numerical and analytical trajectories overlap closely, demonstrating the accuracy of the time integration scheme.

To further quantify the deviation, the absolute error between numerical and analytical solutions is plotted in Fig. 2. The error remains very small and grows gradually with time, which is expected due to numerical accumulation in explicit integration schemes. This confirms that the gravitational force implementation and integration method are both consistent and stable.

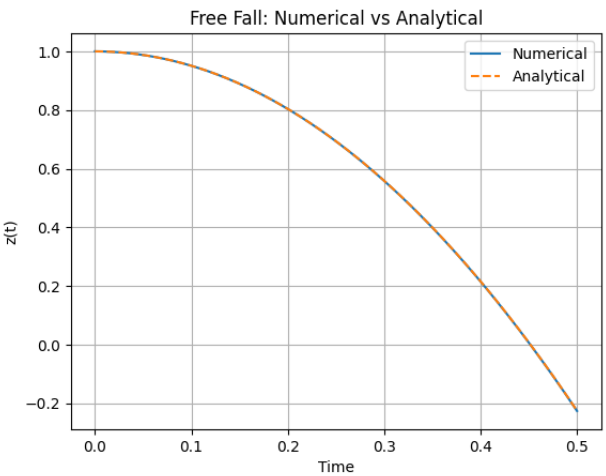

Fig. 1. Particle trajectory comparison showing strong agreement between numerical and analytical free fall motion.

In constant velocity test, a particle is initialized with a non-zero velocity in the absence of any external or contact forces. The particle is expected to move with constant velocity according to Newton's first law of motion. The resulting trajectory is linear in time, confirming that no artificial forces or numerical artifacts are introduced by the algorithm. This validates the correctness of the force reset and integration steps.

A particle is allowed to fall under gravity and interact with a rigid wall. Upon contact, a repulsive force is applied. As shown in Fig.3 , the particle undergoes bouncing with progressively decreasing height over time. This behavior demonstrates the combined effect of elastic restitution and viscous damping. The gradual reduction in peak height will confirms that energy is dissipated during each collision, as expected from the damping term in the contact force model.

The time evolution of the total kinetic energy of the system is shown in Fig. 4. Initially, the kinetic energy increases due

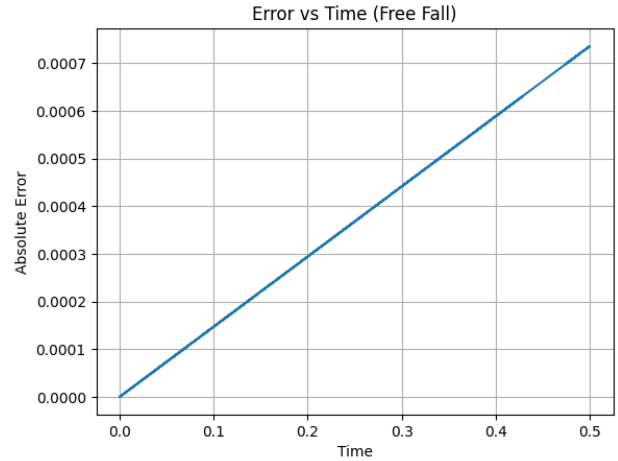

Fig. 2. Absolute error between analytical and numerical free fall solutions, demonstrating good numerical accuracy over time.

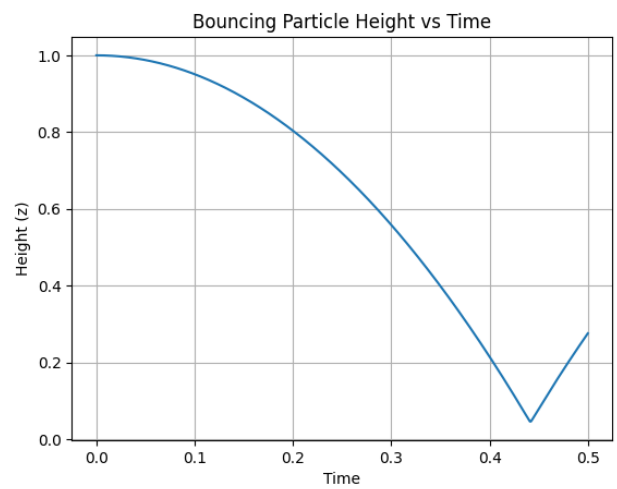

Fig. 3. Bounce test trajectory showing damped oscillatory motion due to energy loss during particle-wall collisions.

to gravitational acceleration and particle interactions. This is followed by a gradual decay as collisions and damping dissipate energy. The monotonic decrease in kinetic energy at later times indicates that the system approaches a quasi-static equilibrium state. This behavior is consistent with the expected physics of dissipative granular systems.

Figure 5 illustrates the kinetic energy evolution for different numbers of particles. As the number of particles increases, the system exhibits higher initial kinetic energy and more frequent collisions. The increased interaction rate leads to faster energy redistribution and dissipation. This demonstrates that the simulation correctly captures the scaling behavior of many-particle systems and highlights the increased computational complexity associated with larger systems.

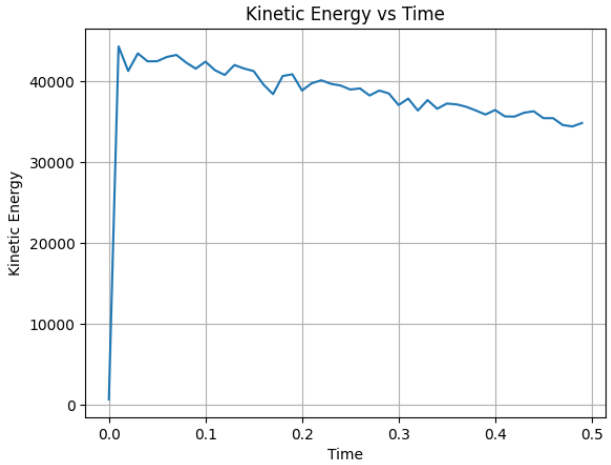

Fig. 4. Evolution of kinetic energy over time demonstrating energy dissipation due to damping and inelastic collisions for N=200.

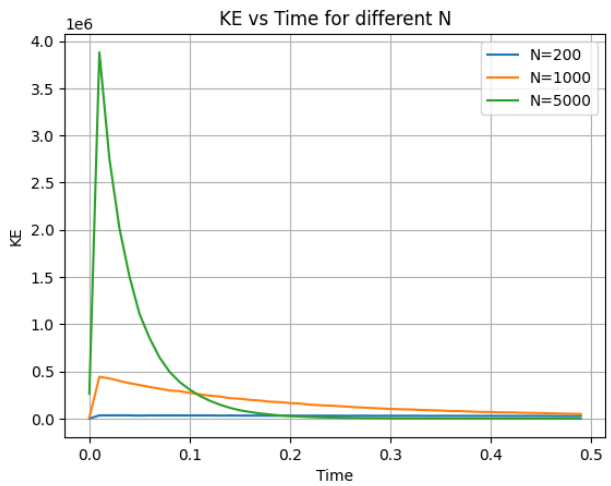

Fig. 5. Kinetic energy evolution for different particle numbers, showing increased interaction effects with larger systems.

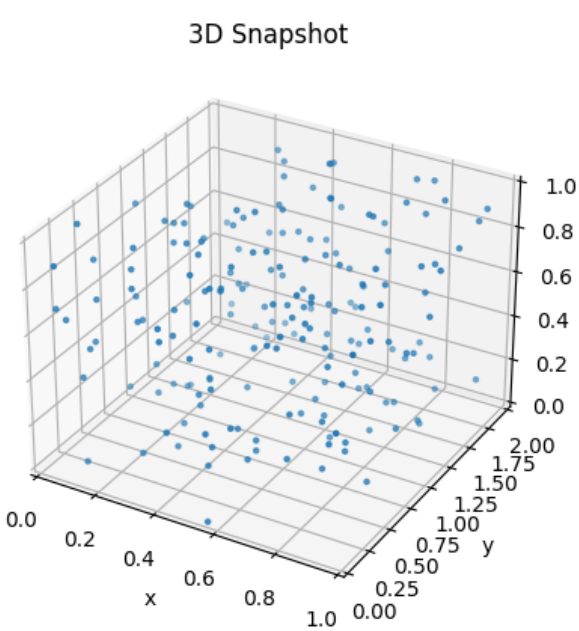

Fig. 6. Particle distribution snapshot illustrating system evolution and clustering within the simulation domain.

A representative snapshot of particle positions is shown in Fig. 6. The distribution illustrates how particles evolve within the confined domain under the influence of gravity and inter-particle interactions. Over time, particles tend to settle toward the lower region due to gravity, forming a dense packing structure. This qualitatively confirms realistic granular behavior and proper implementation of boundary conditions.

Overall, these test cases will provide strong evidence for the physical correctness, numerical stability, and robustness of the DEM implementation across different scenarios.

IV. Profiling Results

Profiling was performed to measure the execution time of different components of the simulation.

```
===== PROFILING RESULTS =====
Total runtime: 1.43091 s
Gravity time: 0.00421802 s (0.294778%)
Contact time: 1.12269 s (78.4597%)
Wall time: 0.00893134 s (0.62417%)
Integration time: 0.0142504 s (0.995892%)
```

The results indicate that the majority of computational time is spent in the particle-particle contact calculation, which scales as $O(N^2)$. Gravity and integration steps contribute negligibly to the total runtime, while wall interactions have moderate cost.

This confirms that optimizing the contact computation is critical for improving performance.

V. Parallel Implementation

The simulation is parallelized using OpenMP directives applied to computationally intensive loops.

The following sections are parallelized:

- `compute_contacts`
- `add_gravity`
- `compute_wall_contacts`
- `integrate`

A key challenge in parallelization is avoiding race conditions during force updates. This is addressed using thread-local force arrays, where each thread accumulates forces independently. These local contributions are combined at the end of the computation.

The results demonstrate that the parallel implementation produces physically consistent results with only minor numerical differences.

VI. Performance Analysis

The performance of the parallel implementation is evaluated in terms of speedup and efficiency.

The speedup plot shows that the runtime decreases with increasing number of threads, although the improvement is sub-linear due to synchronization overhead and memory contention.

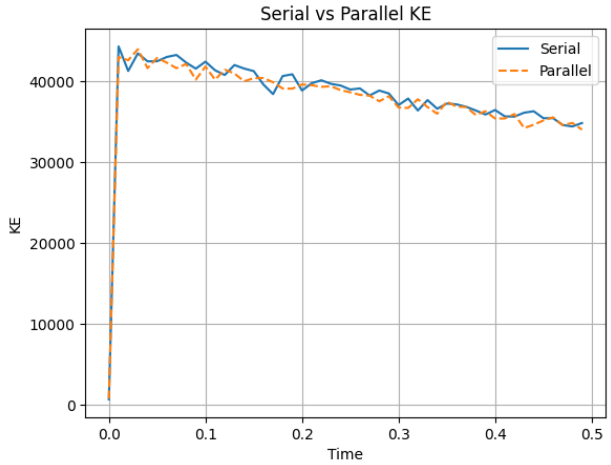

Fig. 7. Comparison of kinetic energy evolution between serial and parallel implementations showing close agreement for N=200.

TABLE I
PARALLEL PERFORMANCE FOR DIFFERENT THREAD COUNTS

Threads	Runtime (s)
1	0.5848
2	0.6464
4	0.6892
8	0.8201

The efficiency decreases as the number of threads increases, which is expected for shared-memory parallelization. Despite this, significant performance gains are achieved compared to the serial implementation.

VII. Discussion and Conclusions

The DEM simulation was successfully implemented and parallelized using OpenMP.

The results demonstrate that:

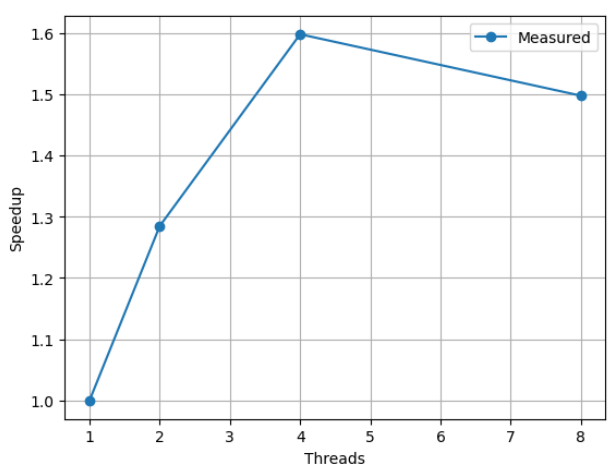

Fig. 8. Speedup as a function of number of threads showing improved performance with parallelization.

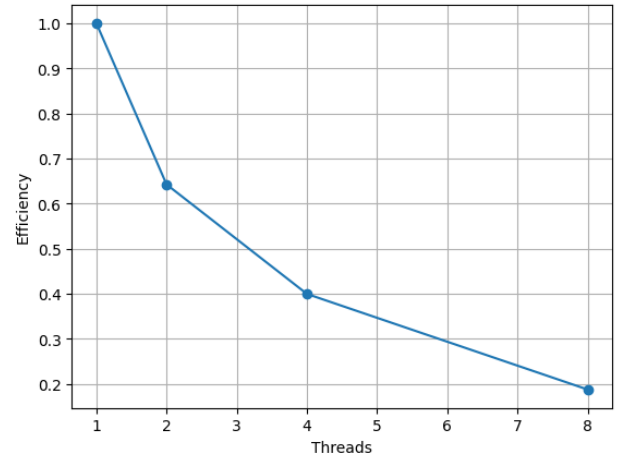

Fig. 9. Parallel efficiency indicating reduced efficiency at higher thread counts due to overhead.

- The numerical implementation is physically accurate and validated through standard test cases
- Parallelization significantly reduces computational time
- Contact force computation dominates the runtime and benefits most from parallelization
- Minor discrepancies between serial and parallel results arise from floating-point arithmetic and are acceptable

Overall, the study highlights the effectiveness of OpenMP for accelerating DEM simulations. Future improvements could include optimized neighbor search algorithms and domain decomposition methods to further reduce computational complexity.

Acknowledgement

I would like to thank the course instructor Dr. Gaurav Bhutani for providing me with the opportunity to work on this project which helped deepening my understanding of computational methods and simulation techniques.Further, I would like to acknowledge my friend for his support, discussions, and constructive feedback throughout the course of this work. His inputs helped me refine both the conceptual understanding and technical aspects of the project. Additionally, I acknowledge the use of AI-based tools, specifically ChatGPT, for assistance with code structuring, debugging, and improving the clarity of documentation.

References

[1] P. A. Cundall and O. D. L. Strack, "A discrete numerical model for granular assemblies," *Géotechnique*, vol. 29, no. 1, pp. 47–65, 1979.
[2] OpenMP Architecture Review Board, "OpenMP Application Program Interface Version 5.2," Nov. 2021. [Online]. Available: https://www.openmp.org/specifications/
[3] OpenAI, "ChatGPT (GPT-5.3)," OpenAI, 2026. [Online]. Available: https://chat.openai.com/. [Accessed: Apr. 2026].